



Build a Virtual Private Cloud (VPC)



Danford Amos

☰ [VPC](#) > [Your VPCs](#) > Create VPC

☒ VPC only ☐ VPC and more

Name tag - *optional*
Creates a tag with a key of 'Name' and a value that you specify.

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me



Introducing Today's Project!

In this project, I will create an Amazon VPC, add a public subnet, and attach an Internet Gateway to enable internet access for resources inside the network. I will demonstrate how core AWS networking components work together and how traffic flows from a private cloud environment to the public internet. I'm doing this project to learn the fundamentals of cloud networking, improve my understanding of VPC architecture, and build a strong foundation for future AWS and DevOps projects.



Virtual Private Clouds (VPCs)

How VPCs work

A VPC is an isolated section of the AWS Cloud that keeps your resources private and secure. Every AWS account includes a default VPC, which exists so you can immediately launch services like EC2 or RDS without building a network from scratch. When creating your own VPC, you must define an IPv4 CIDR block — a range of private IP addresses that AWS will assign to resources inside your network. This CIDR block becomes the foundation for your subnets, routing, and overall VPC architecture.

Why there is a default VPC in AWS accounts

A default VPC was already available in my AWS account because AWS automatically creates one for every new user. This default VPC makes it easy to launch resources like EC2 instances or RDS databases immediately, without having to design a network from scratch. When creating my own custom VPC, I needed to define an IPv4 CIDR block, which specifies the range of private IP addresses the VPC can assign to resources deployed inside it. This CIDR block becomes the foundation for subnet creation, routing, and overall network structure.



☰ [VPC](#) > [Your VPCs](#) > Create VPC

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

Nextwork Project VPC

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Defining IPv4 CIDR blocks

When setting up my VPC, I needed to define an IPv4 CIDR block, which is simply a range of IP addresses that the VPC can allocate to resources deployed inside it. An IPv4 CIDR block uses compact notation to describe how many private IP addresses are available within the network, and it tells AWS exactly which IPs it is allowed to assign to components such as EC2 instances, load balancers, and NAT gateways. My AWS account already included a default VPC—created automatically by AWS so I could launch resources like EC2 or RDS immediately without building a network from scratch—but defining my own CIDR block allowed me to create a custom VPC tailored to my architecture and learning goals.



Subnets

What I did in this step

In this step, I will launch a subnet inside my VPC because a subnet is required to place and organize resources within the network. A subnet is created by carving out a portion of the VPC's IPv4 CIDR block, and it determines where resources like EC2 instances will live and how they will connect to the rest of the network. Creating this subnet is essential because it prepares the VPC for routing, internet access, and future components such as an Internet Gateway or NAT Gateway.

Creating and configuring subnets

Subnets are smaller sections of a VPC that divide its IP address range into organized network segments. Each subnet lives in a single Availability Zone and is used to place resources based on how they should connect to the network. There are already subnets existing in my account, one for every availability zone. My account already had subnets because the default VPC that AWS creates automatically includes one subnet in every Availability Zone in the region that I've set up my VPC in, making it easy to launch resources right away. Since my region is N. Virginia (us-east-1) has 6 Availability Zones. I had 6 default subnets already. I named my subnet Public 1, but that doesn't automatically make my subnet a public subnet. For a subnet to be considered public, it has to be connected to an internet gateway.

Public vs private subnets

Public and private subnets differ based on how they reach the internet. A subnet is considered public when its route table sends 0.0.0.0/0 traffic to an Internet Gateway, allowing resources inside it to have public IPs and communicate directly with the internet. A private subnet does not route traffic to an Internet Gateway; instead, it stays internal and typically uses a NAT Gateway for outbound-only access.



Subnets (1/7) [Info](#) Last updated 12 minutes ago [Actions](#) [Create subnet](#)

☐	Name	Subnet ID	State	VPC
☐	-	subnet-0b1e3b24b2043f458	Available	vpc-091bb2fe0f0f31608
☐	-	subnet-089314321e9333917	Available	vpc-091bb2fe0f0f31608
☐	-	subnet-0821036fa09b7f5ab	Available	vpc-091bb2fe0f0f31608
☐	-	subnet-0bef53b5177dac363	Available	vpc-091bb2fe0f0f31608
☒	Public 1	subnet-05daef1f39b3f3e87	Available	vpc-0cd08f4020290bf9d Next...

Auto-assigning public IPv4 addresses

Once I created my subnet, I enabled auto-assign public IPv4 addresses because this setting ensures that any EC2 instance launched in the subnet automatically receives a public IP. This matters because a subnet is only useful for internet-facing resources if those resources can be reached from outside the VPC. Enabling this option saves time, prevents misconfiguration, and guarantees that instances in my public subnet can communicate with the internet through the Internet Gateway without needing to manually assign public IPs each time.



Internet gateways

What I did in this step

In this step, I will attach my VPC to an Internet Gateway because the Internet Gateway is what allows resources inside a public subnet to communicate with the internet. A VPC by itself is completely isolated, so attaching an Internet Gateway creates the path that outbound traffic can use to reach external networks and enables inbound traffic for resources that have public IPs. This attachment is a required step before configuring the route table that will make my subnet function as a true public subnet.

Setting up internet gateways

An Internet Gateway is simply the component that gives your VPC a path to the internet. It lets resources in a public subnet send traffic out to the internet and receive traffic back, but only if the subnet's route table points to it. Without an Internet Gateway, your VPC stays completely isolated, even if instances have public IPs.

Attaching an Internet Gateway to your VPC means giving the VPC a path to the internet. Without it, the VPC stays fully isolated. So if I missed this step, any instance launched in the subnet would fail to reach the internet, and inbound connections would fail too, because there would be no external route for traffic to enter or leave the VPC.



igw-056af54030f3c256a

Internet gateway igw-056af54030f3c256a successfully attached to vpc-0cd08f4020290bf9d

igw-056af54030f3c256a / NextWork IG

Actions

Details

Info

Internet gateway ID

igw-056af54030f3c256a

State

Attached

VPC ID

[vpc-0cd08f4020290bf9d](#)
[Nextwork Project VPC](#)

Owner

[171414227038](#)

Tags (1)

Manage tags

Search tags

< 1 > ⚙

Key

Value

Name

NextWork IG



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

